TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT NO. 1

THREE CREATIONAL PATTERNS IN A TASK REPORTING PROJECT

Student: Lucian-Adrian Gavril

Group: FAF-241

Coordinator: asist. univ., Daniel Istrati

Chișinău, 2026

TABLE OF CONTENTS

ABSTRACT 3

INTRODUCTION 4

1 SELECTED PATTERNS AND IMPLEMENTATION 5

1.1 Builder: collect report options 5

1.2 Factory Method: choose one report formatter 5

1.3 Abstract Factory: create a matching board style 5

2 SOURCE-LEVEL TRACE AND RESULTS 6

2.1 Filter the input tasks 6

2.2 Create the two outputs 6

2.3 Build and run 7

2.4 Questions for oral review 7

CONCLUSIONS 8

REFERENCES 9

APPENDICES 10

APPENDIX A – Source files 10

A.1 Client 10

A.2 Shared model 12

A.3 Creational types 13

ABSTRACT

This report explains the Builder, Factory Method, and Abstract Factory patterns in a task-reporting program. The builder collects a title and minimum priority into one options value. The program filters a task list, then uses a report creator to format the selected tasks as CSV. A second component uses an abstract factory to create a matching Markdown heading and task-row formatter. The report traces the input through these components, maps each pattern to its implementation, and gives build commands and oral-review questions. The same project includes the behavioral continuation in Laboratory 3 and uses one command-line client for both demonstrations.

Keywords: creational patterns, builder, factory method, abstract factory, task report

INTRODUCTION

The TMPS Laboratory 1 assignment asks for three creational patterns from its list and familiarity with the others. It marks Object Pooling as useful to know rather than required [1, 2]. This project selects Builder, Factory Method, and Abstract Factory.

The example is a small task-reporting program. Its objectives are:

a) explain how the builder turns a sequence of settings into one options value;

b) trace how a creator chooses a CSV formatter and how a theme factory creates a matching Mark-down header and row;

c) follow the three input tasks through the filters and connect the result to the client and shared model in Listings A.1 and A.2.

1 SELECTED PATTERNS AND IMPLEMENTATION

The course's recommended references include the standard design-pattern catalog by Gamma, Helm, Johnson, and Vlissides [3]. This chapter maps three choices from the assignment list to their concrete roles in the client and pattern files shown in Listings A.1 and A.3.

1.1 Builder: collect report options

`TaskReportOptionsBuilder` holds a title, an include-completed flag, and a minimum priority. Each setter changes one setting and returns the builder, so the calls can be chained. `Build` copies the chosen values into one `TaskReportOptions` record.

In `RunLab01`, the program sets the title to `Study tasks` and the minimum priority to 2. The option to include completed tasks remains false, which is the builder's default. The result is a single configuration value passed to the task selector and the output components.

This keeps several options together instead of passing separate parameters through each call. The example does not validate the title or priority in `Build`; a production version would need to decide what values are valid.

1.2 Factory Method: choose one report formatter

`TaskReportCreator.Build` asks its subclass for a formatter, then delegates the output.

`CsvTaskReportCreator` creates the CSV formatter.

`TextTaskReportCreator` creates the plain-text formatter.

`RunLab01` constructs the CSV creator. The same build workflow can use a different formatter by choosing another creator subtype. The pattern is useful here because the creation decision belongs to the creator subclass, while formatting behavior stays in the formatter.

1.3 Abstract Factory: create a matching board style

`IBoardThemeFactory` creates two related products: an `IBoardHeader` and an `IBoardTaskRow`. `MarkdownBoardThemeFactory` returns Markdown versions of both. `PlainTextBoardThemeFactory` returns plain-text versions.

`TaskBoardRenderer` receives the factory, stores the two products, and uses them to render a title and rows. The program selects the Markdown factory, keeping the heading and row formats consistent. The abstract factory is useful because the renderer depends on interfaces and does not construct a mismatched header and row itself. Laboratory 3 reuses this renderer in its behavioral client.

2 SOURCE-LEVEL TRACE AND RESULTS

This chapter follows the exact task values and calls in `Program.RunLab01`. It is a source-level trace rather than a performance test. The complete client, shared task model, and creational types appear in Listings A.1, A.2, and A.3.

2.1 Filter the input tasks

The program starts with three shared `TaskItem` objects. The first filter reads `IncludeCompleted`, which remains false. It removes task 3 because that task is marked done.

The second filter keeps tasks whose priority is at least 2. The result preserves the original order of the two selected records.

Table 2.1 – Input tasks and selection result

ID	Task	Priority	Initial state	Selected
1	Write report	3	Open	Yes
2	Review notes	2	Open	Yes
3	Archive old draft	1	Done	No

Task 3 is excluded by the completed-task filter. The minimum-priority filter then retains task 1 at priority 3 and task 2 at priority 2. No sorting call is made, so the output keeps their initial order.

2.2 Create the two outputs

The CSV creator receives the title and selected tasks. Its formatter writes the title, a header row, then one row per selected task. The theme factory independently supplies the Markdown header and task-row implementations used by `TaskBoardRenderer`. The code therefore demonstrates one Factory Method decision for one formatter and one Abstract Factory decision for a family of two related rendering products.

The source predicts these outputs:

```
Study tasks
Id,Title,Priority,Status
1,"Write report",3,Open
2,"Review notes",2,Open

# Study tasks
- [ ] Write report (P3)
- [ ] Review notes (P2)
```

The complete build and run commands appear in the project README and are repeated below for convenient review. The source-level trace does not claim benchmark results.

2.3 Build and run

From the `lab-01` directory, use the .NET 10 SDK. Build artifacts are directed outside the project folder:

```
$build = Join-Path $env:TEMP 'tmps-build'
dotnet build .\TMPSLab01.csproj -c Release --artifacts-path $build
dotnet (Join-Path $build 'bin\TMPSLab01\release\TMPSLab01.dll') lab-01
```

The build command compiles both laboratory modules into one executable. The `lab-01` argument selects this report's demonstration. The program prints the selected tasks first in CSV form and then in Markdown form. The plain-text report creator and plain-text board factory are implemented but are not selected by `RunLab01`.

2.4 Questions for oral review

1. What values does `TaskReportOptionsBuilder` collect? It stores a title, whether completed tasks are included, and a minimum priority.
2. What decision does Factory Method make in this example? A creator subclass chooses the formatter returned by its creation method.
3. What makes the theme factory abstract? It supplies a matching header and task-row pair through interfaces.
4. How can the program include completed tasks? Call `IncludeCompletedTasks` while building the options.
5. How would a JSON report fit the existing design? Add a JSON formatter and a creator subtype that returns it.

CONCLUSIONS

The example applies three creational patterns to separate decisions about configuration and object creation. Builder assembles one options value. Factory Method chooses the report formatter. Abstract Factory creates a matching pair of board renderers. The source trace shows that the selected settings remove the completed low-priority task and preserve the order of the two open tasks.

The example has limits: the builder does not validate settings, the sample uses in-memory data, and this mode exercises only one creator and one theme factory at runtime. The shared executable also contains the behavioral continuation, which uses the same task model and creational components.

REFERENCES

1. Technical University of Moldova, ELSE. *FAF.TMPS21.1: Laboratory work #1*, Creational Design Patterns. Available online: `https://else.fcim.utm.md/mod/assign/view.php?id=27825` (accessed 10 October 2026).

2. V. Drumea. *SDTM-Labs, Lab #1 README*. Project organization, explanation, and repository submission requirements. Available online: `https://github.com/DrVasile/SDTM-Labs/blob/master/Lab%231/README.md` (accessed 10 October 2026).

3. E. Gamma, R. Helm, R. Johnson, and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. The course lists this work in its references section.

APPENDICES

APPENDIX A – Source files

The following files show the single command-line client, shared task model, and creational pattern types described in this report. The project also compiles `BehavioralPatterns.cs` from the Laboratory 3 folder into the same assembly.

A.1 Client

Listing A.1: Single client with Lab 1 and Lab 3 modes

```
using System;
using System.Collections.Generic;
using System.Linq;
using TMPS.Behavioral;
using TMPS.Creational;
using TMPS.Domain;

namespace TMPS.Client;

internal static class Program
{
    private static int Main(string[] args)
    {
        if (args.Length == 0 || args[0] == "lab-01")
        {
            RunLab01();
            return 0;
        }

        if (args.Length == 1 && args[0] == "lab-03")
        {
            RunLab03();
            return 0;
        }

        Console.Error.WriteLine("Usage: TMPSLab01 [lab-01|lab-03]");
        return 1;
    }

    private static void RunLab01()
```

```
{
    List<TaskItem> allTasks = new()
    {
        new TaskItem(1, "Write report", 3, new DateOnly(2026, 10, 12), false),
        new TaskItem(2, "Review notes", 2, new DateOnly(2026, 10, 11), false),
        new TaskItem(3, "Archive old draft", 1, new DateOnly(2026, 10, 10), true)
    };

    TaskReportOptions options = new TaskReportOptionsBuilder()
        .WithTitle("Study tasks")
        .WithMinimumPriority(2)
        .Build();

    List<TaskItem> selectedTasks = SelectTasks(allTasks, options);

    TaskReportCreator reportCreator = new CsvTaskReportCreator();
    Console.WriteLine(reportCreator.Build(options.Title, selectedTasks));

    TaskBoardRenderer board = new(new MarkdownBoardThemeFactory());
    Console.WriteLine(board.Render(options.Title, selectedTasks));
}

private static void RunLab03()
{
    TaskBoard board = new(new List<TaskItem>
    {
        new(1, "Write report", 3, new DateOnly(2026, 10, 12), false),
        new(2, "Review notes", 2, new DateOnly(2026, 10, 11), false),
        new(3, "Archive old draft", 1, new DateOnly(2026, 10, 13), false)
    });

    TaskReportOptions options = new TaskReportOptionsBuilder()
        .WithTitle("Study tasks")
        .WithMinimumPriority(1)
        .Build();
    ITaskSortStrategy sortStrategy = new PrioritySortStrategy();

    board.Subscribe(new ConsoleTaskObserver());

    List<TaskItem> tasksBeforeCommand = SelectTasks(board.GetTasks(sortStrategy),
options);
    TaskReportCreator reportCreator = new CsvTaskReportCreator();
```

```
        Console.WriteLine(reportCreator.Build(options.Title, tasksBeforeCommand));

        board.Execute(new CompleteTaskCommand(2));

        List<TaskItem> openTasks = SelectTasks(board.GetTasks(sortStrategy), options)
    ;
        TaskBoardRenderer boardRenderer = new(new MarkdownBoardThemeFactory());
        Console.WriteLine(boardRenderer.Render(options.Title, openTasks));
    }

    private static List<TaskItem> SelectTasks(
        IEnumerable<TaskItem> tasks,
        TaskReportOptions options)
    {
        return tasks
            .Where(task => options.IncludeCompleted || !task.IsCompleted)
            .Where(task => task.Priority >= options.MinimumPriority)
            .ToList();
    }
}
```

A.2 Shared model

Listing A.2: Task model shared by both laboratories

```
using System;

namespace TMPS.Domain;

internal sealed class TaskItem
{
    public TaskItem(int id, string title, int priority, DateOnly dueDate, bool
    isCompleted)
    {
        Id = id;
        Title = title;
        Priority = priority;
        DueDate = dueDate;
        IsCompleted = isCompleted;
    }

    public int Id { get; }
    public string Title { get; }
```

```
    public int Priority { get; }
    public DateOnly DueDate { get; }
    public bool IsCompleted { get; private set; }

    public void MarkComplete()
    {
        IsCompleted = true;
    }
}
```

A.3 Creational types

Listing A.3: Builder, Factory Method, and Abstract Factory

```
using System;
using System.Collections.Generic;
using TMPS.Domain;

namespace TMPS.Creational;

internal sealed record TaskReportOptions(
    string Title,
    bool IncludeCompleted,
    int MinimumPriority);

// Builder: set optional report settings one step at a time.
internal sealed class TaskReportOptionsBuilder
{
    private string _title = "Task report";
    private bool _includeCompleted;
    private int _minimumPriority = 1;

    public TaskReportOptionsBuilder WithTitle(string title)
    {
        _title = title;
        return this;
    }

    public TaskReportOptionsBuilder IncludeCompletedTasks()
    {
        _includeCompleted = true;
        return this;
    }
```

```
    public TaskReportOptionsBuilder WithMinimumPriority(int priority)
    {
        _minimumPriority = priority;
        return this;
    }

    public TaskReportOptions Build()
    {
        return new TaskReportOptions(_title, _includeCompleted, _minimumPriority);
    }
}

internal interface ITaskReportFormatter
{
    string Format(string title, IEnumerable<TaskItem> tasks);
}

// Factory Method: subclasses choose which formatter to create.
internal abstract class TaskReportCreator
{
    public string Build(string title, IEnumerable<TaskItem> tasks)
    {
        ITaskReportFormatter formatter = CreateFormatter();
        return formatter.Format(title, tasks);
    }

    protected abstract ITaskReportFormatter CreateFormatter();
}

internal sealed class CsvTaskReportCreator : TaskReportCreator
{
    protected override ITaskReportFormatter CreateFormatter()
    {
        return new CsvTaskReportFormatter();
    }
}

internal sealed class TextTaskReportCreator : TaskReportCreator
{
    protected override ITaskReportFormatter CreateFormatter()
    {
```

```
        return new TextTaskReportFormatter();
    }
}

internal sealed class CsvTaskReportFormatter : ITaskReportFormatter
{
    public string Format(string title, IEnumerable<TaskItem> tasks)
    {
        List<string> lines = new() { title, "Id,Title,Priority,Status" };

        foreach (TaskItem task in tasks)
        {
            string status = task.IsCompleted ? "Done" : "Open";
            lines.Add($"{task.Id},{Escape(task.Title)},{task.Priority},{status}");
        }

        return string.Join(Environment.NewLine, lines);
    }

    private static string Escape(string value)
    {
        return "\"" + value.Replace("\"", "\"\"") + "\"";
    }
}

internal sealed class TextTaskReportFormatter : ITaskReportFormatter
{
    public string Format(string title, IEnumerable<TaskItem> tasks)
    {
        List<string> lines = new() { title };

        foreach (TaskItem task in tasks)
        {
            string status = task.IsCompleted ? "done" : "open";
            lines.Add($"{task.Id}. {task.Title} (priority {task.Priority}, {status})
   ");
        }

        return string.Join(Environment.NewLine, lines);
    }
}
```

```
internal interface IBoardHeader
{
    string Format(string title);
}

internal interface IBoardTaskRow
{
    string Format(TaskItem task);
}

internal interface IBoardThemeFactory
{
    IBoardHeader CreateHeader();
    IBoardTaskRow CreateTaskRow();
}

// Abstract Factory: create a matching family of header and row formatters.
internal sealed class PlainTextBoardThemeFactory : IBoardThemeFactory
{
    public IBoardHeader CreateHeader() => new PlainTextBoardHeader();
    public IBoardTaskRow CreateTaskRow() => new PlainTextBoardTaskRow();
}

internal sealed class MarkdownBoardThemeFactory : IBoardThemeFactory
{
    public IBoardHeader CreateHeader() => new MarkdownBoardHeader();
    public IBoardTaskRow CreateTaskRow() => new MarkdownBoardTaskRow();
}

internal sealed class PlainTextBoardHeader : IBoardHeader
{
    public string Format(string title) => title;
}

internal sealed class PlainTextBoardTaskRow : IBoardTaskRow
{
    public string Format(TaskItem task)
    {
        string status = task.IsCompleted ? "done" : "open";
        return $"{task.Id}. {task.Title} (priority {task.Priority}, {status})";
    }
}
```

```
internal sealed class MarkdownBoardHeader : IBoardHeader
{
    public string Format(string title) => $"# {title}";
}

internal sealed class MarkdownBoardTaskRow : IBoardTaskRow
{
    public string Format(TaskItem task)
    {
        string marker = task.IsCompleted ? "x" : " ";
        return $"- [{marker}] {task.Title} (P{task.Priority})";
    }
}

internal sealed class TaskBoardRenderer
{
    private readonly IBoardHeader _header;
    private readonly IBoardTaskRow _taskRow;

    public TaskBoardRenderer(IBoardThemeFactory factory)
    {
        _header = factory.CreateHeader();
        _taskRow = factory.CreateTaskRow();
    }

    public string Render(string title, IEnumerable<TaskItem> tasks)
    {
        List<string> lines = new() { _header.Format(title) };

        foreach (TaskItem task in tasks)
        {
            lines.Add(_taskRow.Format(task));
        }

        return string.Join(Environment.NewLine, lines);
    }
}
```